

APP DEVELOPMENT II PROJECT REPORT

Placement Portal Application - V2

1. Author

Abhishek Tiwari

Roll Number: 24F2006100

Email: 24f2006100@ds.study.iitm.ac.in

I am a student of the IIT Madras online degree programme with an interest in application development, data science, and machine learning. This project helped me apply full-stack development concepts by integrating a Vue.js frontend with a Flask API, database management, caching, and background jobs.

2. AI/LLM Usage

I used ChatGPT (GPT-5) to assist me in this project. The extent of AI/LLM usage is around 10–15%, limited to code suggestions and documentation formatting. All final implementation logic, debugging, and integration were done manually.

3. Description

The objective of the project is to build a multi-user placement management application for an institute. The system provides separate role-based portals for the Admin, Companies, and Students to manage company approvals, placement drives, applications, selection status, resumes, reports, and notifications.

4. Technologies Used

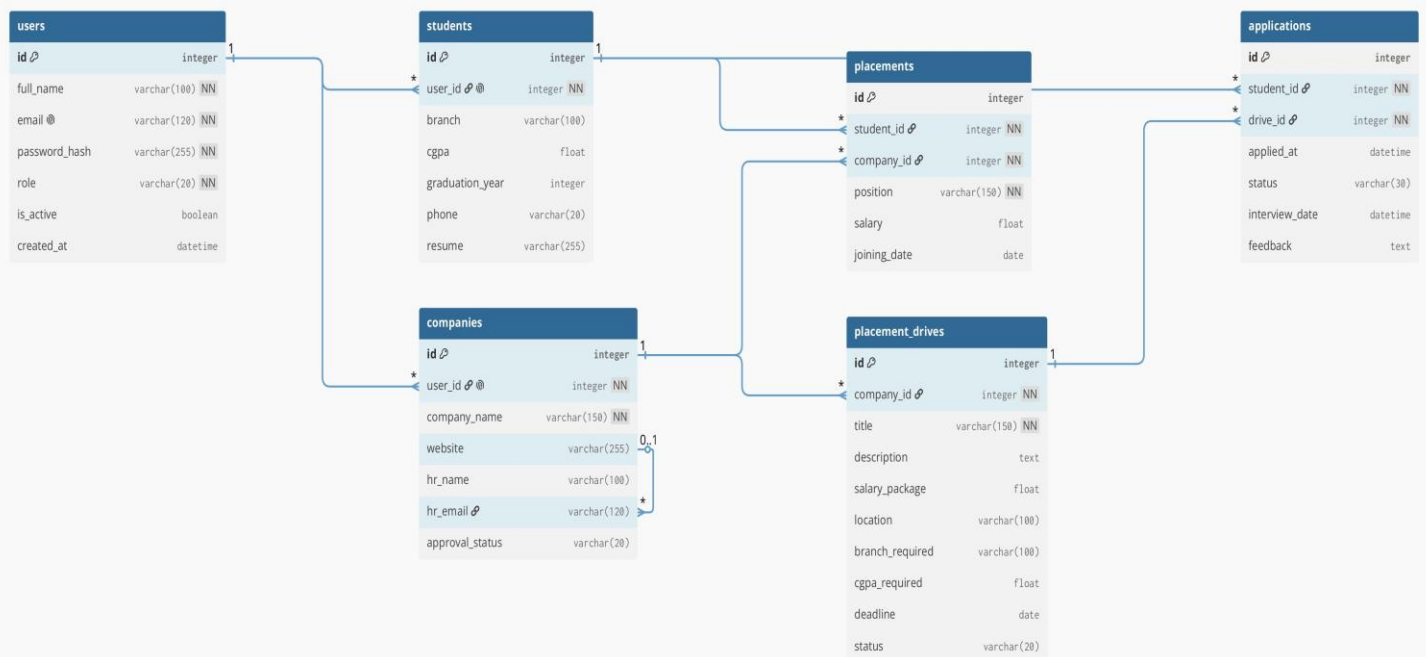
Technology	Purpose
Flask	Backend framework and REST APIs
Flask-SQLAlchemy / SQLAlchemy	Database models, relationships, and operations
SQLite	Relational application database
Vue.js + Vite	Frontend user interface
Axios	Frontend-backend API communication
Flask-JWT-Extended	JWT authentication and role-based access control
Flask-Caching	Caching integration for API responses
Redis / Memurai	Caching and Celery broker/result backend
Celery + Celery Beat	Asynchronous and scheduled background jobs
Flask-CORS	Communication between Vue and Flask
Werkzeug	Password hashing and verification
Gmail SMTP	Reminder and monthly report emails

5. DB Schema Design

The database is normalized into six main tables. A unified Users table stores common authentication and role information, while Students and Companies store role-specific profile data. Placement drives belong to companies, applications connect students with placement drives, and completed placement information is stored separately in the Placements table.

Table	Columns / Constraints
users	id (PK), full_name, email (UNIQUE), password_hash, role, is_active, created_at
students	id (PK), user_id (FK, UNIQUE), branch, cgpa, graduation_year, phone, resume
companies	id (PK), user_id (FK, UNIQUE), company_name, website, hr_name, hr_email, approval_status
placement_drives	id (PK), company_id (FK), title, description, salary_package, location, branch_required, cgpa_required, deadline, status
applications	id (PK), student_id (FK), drive_id (FK), applied_at, status, interview_date, feedback
placements	id (PK), student_id (FK), company_id (FK), position, salary, joining_date

The design separates common user credentials from role-specific information to avoid duplication. Foreign keys maintain relationships between users, companies, students, drives, applications, and placements. Unique constraints preserve identity consistency, while duplicate-application prevention and CGPA eligibility are enforced through backend logic.



6. API Design

The application uses REST APIs implemented through Flask Blueprints. Authentication APIs handle login and student/company registration. Admin APIs provide dashboard statistics, approvals, search, deactivation, application viewing, and exports. Company APIs manage placement drives and applicant workflows. Student APIs provide approved-drive search, CGPA-validated applications, profile and resume management, application history, and asynchronous CSV export. Protected endpoints use JWT authentication and role checks.

The API definition YAML file is submitted separately.

7. Architecture and Features

The project follows a separated frontend-backend architecture. The Flask backend is organized into route controllers using Blueprints for authentication, Admin, Company, and Student operations. SQLAlchemy models are maintained separately from route logic. The Vue.js frontend contains reusable components, routing, API services, and role-specific view pages. Axios connects the frontend to the Flask APIs. Redis supports caching and Celery, while Celery Beat schedules recurring background jobs.

- JWT-based role authentication for Admin, Company, and Student users.
- Admin approval/rejection of companies and placement drives, search, deactivation, and application monitoring.
- Company drive management and applicant processing, including resume viewing, shortlisting, interviews, offers, feedback, and placement status.
- Student drive search, CGPA eligibility validation, duplicate-application prevention, resume upload/view, profile editing, and placement history.
- Redis caching with expiry and invalidation after important data changes.
- Daily deadline reminder emails and a monthly HTML placement activity report using Celery Beat.
- User-triggered asynchronous CSV export of student application history and Admin data exports.

8. Video

Video Presentation Link:

<https://drive.google.com/file/d/1pnSCL5M5TFB-61q1qqvqaGgCRVLg24ca/view?usp=sharing>